

AI-based Village Pond Planning System

CSD Assignment 1 – Final Technical Report

KHETHAVATH SUNIL NAIK

12341170

Selecting sites for village ponds is traditionally a manual, experience-driven task: planners study paper contour sheets, guess where runoff converges, and hope seasonal rainfall justifies the excavation cost. This report presents PondSite, a full-stack web application that makes pond-site selection explainable and data-driven. A React single-page front end lets a planner drop a study circle on a Leaflet map (or upload a KML/KMZ contour sheet); a Node/Express backend then builds a real digital elevation model from live elevation APIs (Copernicus DEM GLO-90 via Open-Meteo, with SRTM 30 m via OpenTopoData as second provider), fetches three years of ERA5 rainfall and OpenStreetMap water/road/building context via Overpass, and runs sink-filling, D8 flow routing, flow accumulation and Strahler ordering to delineate upstream catchments. Candidate sites are hard-filtered to a village-pond catchment band of 1,000–6,000 m², excluded inside mapped water, scored on four transparent weighted factors, and returned with per-site runoff ($R = P \cdot A \cdot C$, $C=0.45$), terrain-derived storage and plain-language explanations. Every number comes from live data: if an external API fails, the system returns an honest error instead of inventing results. The system was verified end-to-end: a 500 m study circle produced five band-compliant candidates, and a 1 m-contour KML sheet produced 14 candidates with catchments of 3,201–3,784 m², all within the band, with typical end-to-end analyses completing in under a minute.

Additional Key Words and Phrases: Village Pond Planning, Geospatial Analysis, Rainwater Harvesting, Catchment Estimation, Web GIS, REST APIs

ACM Reference Format:

Khethavath Sunil Naik, 12341170. 2026. AI-based Village Pond Planning System: CSD Assignment 1 – Final Technical Report. 1, 1 (September 2026), 13 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

Village ponds are small, low-cost reservoirs that store monsoon runoff for drinking water for livestock, recharge of shallow wells, and supplementary irrigation. Their effectiveness depends almost entirely on *where* they are dug: a pond placed on a natural flow line with a modest upstream catchment fills every season, while the same pond placed a hundred metres away on a spur may never hold water. Despite this sensitivity, site selection in most villages is still done by eye, on paper maps, from memory of where water stood in past years.

This report describes PondSite, a web application built for CSD Assignment 1 that recommends candidate pond locations from live geospatial data and explains every recommendation. The planner draws a study circle around a village (or uploads a KML contour sheet), and the system samples a real digital elevation model (DEM), estimates the

Author's Contact Information: Khethavath Sunil Naik
12341170, khethavathn@iitbhilai.ac.in.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Manuscript submitted to ACM

upstream catchment of every point, combines it with three years of observed rainfall and OpenStreetMap (OSM) ground context, and returns 5–10 ranked, scored, and annotated candidate sites.

The rest of the report is organised as follows. Section 2 restates the functional and non-functional requirements and maps each to the module that implements it. Section 3 presents the system architecture and technology stack. Section 4 details the terrain, catchment and rainfall algorithms. Section 5 covers the REST API, the map front end, and storage decisions. Section 6 maps the implementation to core CSD course themes. Section 7 reports verified results and performance, Section 8 discusses limitations honestly, and Section 9 declares AI tool usage. The appendix documents the repository layout.

1.1 Motivation

Manual pond-site selection is hard for structural reasons, not just resource reasons:

- **Terrain complexity.** Judging where runoff converges requires reading contour lines across an entire catchment; a single wrong reading sends the pond to a spur instead of a valley line.
- **Scattered data.** Rainfall records sit with meteorological agencies, elevation with survey agencies, and water bodies with the village itself. No single office assembles them, so decisions are made from one source at best.
- **No auditability.** A hand-picked site comes with no written justification. When a pond fails to fill, nobody can reconstruct why the spot was chosen.
- **Existing-water conflicts.** New ponds are sometimes proposed on top of existing tanks or seasonal water bodies because the paper map is out of date.

An automated tool addresses each of these: it assembles live terrain, rainfall and OSM data for the requested area, applies a consistent hydrological procedure, and attaches a human-readable explanation (scoring factors, cautions, catchment statistics) to every candidate. Village administrators get a shortlist they can verify on the ground, not a black-box answer.

1.2 Scope of the Project

PondSite *does*:

- analyse a user-selected circular study area (radius 100–20,000 m) using a live DEM, historical rainfall, and OSM water bodies, waterways, roads and buildings;
- alternatively analyse an uploaded KML/KMZ contour map (1 m contour sheets work well), optionally restricted to a study circle;
- estimate the upstream catchment area of each candidate via D8 flow routing and flow accumulation;
- query historical rainfall (Open-Meteo ERA5 archive, 3-year monthly means) and estimate annual runoff volume into each candidate ($R = P \cdot A \cdot C$ with a fixed runoff coefficient $C = 0.45$);
- recommend a terrain-derived pond footprint and storage volume at a design depth (default 3 m) and rank candidates with an explainable weighted score;
- hard-exclude candidates inside mapped OSM water bodies;
- export the full analysis as JSON.

PondSite *does not*:

- perform structural or civil engineering design of the pond (embankment dimensions, seepage/lining design, spillway sizing);

- consult land-ownership or revenue records – the tool reasons about terrain and mapped features only;
- model groundwater recharge, soil infiltration, or water quality;
- guarantee completeness of OSM data, which varies by region.

2 Problem Statement and Requirements

The assignment asked for a web application that helps village planners identify suitable pond sites: the user must be able to view satellite imagery of the area, overlay contour information, identify available land, estimate the catchment contributing runoff to a candidate point, query historical rainfall, estimate runoff volume, and receive a pond depth/storage recommendation, all in a combined results view. Table 1 maps each requirement to the module that satisfies it in this implementation.

Table 1. Functional requirements and where they are implemented

Requirement	Implemented in
Satellite imagery display	PondMap.tsx (Leaflet + Esri World Imagery base map)
Contour map visualization	PondMap.tsx + contour.ts (KML parse, polyline overlay)
Available-land identification	overpass.ts + analyze-circle.ts (OSM water/building exclusion)
Catchment area estimation	contour.ts (D8 + upstreamCells), analyze-contour.ts
Historical rainfall query	rainfall.ts (Open-Meteo ERA5 archive)
Runoff volume estimation	analyze-circle.ts ($R = P \cdot A \cdot C$, $C=0.45$ fixed in App.tsx)
Pond depth / storage recommendation	analyze-contour.ts (pondForCatchment: footprint + volume at design depth)
Combined overlay / results view	PondMap.tsx + Workspace.tsx + Report.tsx

2.1 Non-Functional Requirements

The following non-functional requirements were set and met:

- **Responsiveness.** A full analysis (terrain + rainfall + OSM) should complete in under a minute on a normal residential connection. This drove the decision to fetch the three external datasets in parallel and to race all Overpass mirrors concurrently (Section 6).
- **Robustness to flaky external APIs.** Each DEM/rainfall batch is retried up to three times with backoff; requests carry 30 s timeouts (60 s for Overpass); seven Overpass mirrors are tried so a single blocked host cannot stall an analysis.
- **Honest degradation.** If mandatory data (terrain, rainfall) is unavailable the API must return an explicit error – it must never fabricate results. If optional data (OSM) is unreachable the response must say so explicitly (waterExclusion: {applied: false}).

- **Input validation.** Coordinates, radius (100–20,000 m) and uploads (only .kml/.kmz, ≤ 40 MB) are validated server-side; bad input returns HTTP 400 with a specific message.
- **Usability.** The map must remain responsive during analysis; the health of the backend is surfaced by a status pill polled every 30 s.
- **Browser support.** Evergreen browsers (Chrome, Edge, Firefox); no plugins required.
- **Security posture.** The service exposes no user data and no write operations; it is intended for trusted deployment (course/intranet scope). CORS is enabled for local development.

3 System Architecture and High-Level Design

PondSite is a two-tier system: a stateless Express backend that performs all analysis, and a React single-page application that renders results. There is no server-side database; all persistence is client-side (Section 5). All heavy data – elevation, rainfall, OSM context – is fetched live from external public APIs at analysis time.

Figure 1 shows the data flow. The browser sends a single JSON request (POST /api/analyze with latitude, longitude and radius, or POST /api/analyzeContour with a KML/KMZ upload and an optional circle). The backend then fans out *in parallel* to three external data sources: the elevation APIs (Copernicus GLO-90 via Open-Meteo, falling back to SRTM 30 m via OpenTopoData), the Open-Meteo ERA5 archive for rainfall, and the Overpass API (seven mirrors raced) for OSM water, roads and buildings. The analysis pipeline (DEM construction → sink filling → D8 flow routing → flow accumulation and Strahler ordering → candidate scoring and exclusion) runs on the merged data, and one response containing the DEM metadata, rainfall statistics, water layers and the ranked candidate list returns to the browser, which draws the overlay on the map and the breakdown cards beside it.

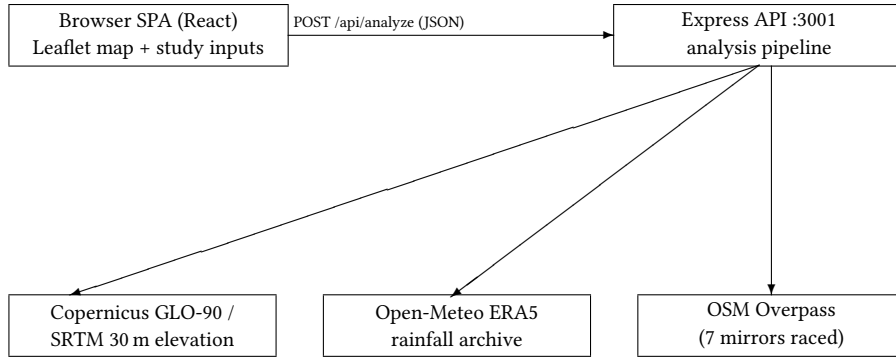


Fig. 1. High-level system architecture. All three external datasets are fetched in parallel for every analysis; terrain and rainfall are mandatory (failure aborts with an honest error) while OSM is optional and reported as not-applied when unreachable.

The candidate pipeline inside the backend proceeds in six stages: (1) sample the live DEM over the study area (~20 m cells, capped at 64×64 points); (2) fetch rainfall and OSM context concurrently with the DEM; (3) run sink filling, D8 flow direction, flow accumulation, slope and local-relief computation; (4) score every in-band grid cell (flow convergence, slope, concavity, data support) and apply non-maximum suppression; (5) delineate the upstream catchment of each surviving cell and apply hard exclusions (inside mapped water, out-of-band catchment, slope ceiling); and (6) compute runoff, pond footprint, storage, explanations and the final radius-dependent candidate cap (5–10).

3.1 Technology Stack

The implemented stack is:

- **Language:** TypeScript end-to-end (strict mode, shared geometry and type modules between front end and back end).
- **Backend:** Node.js with Express 4, multer for multipart uploads, fflate for KMZ unzipping, run with tsx.
- **Frontend:** React 19 + Vite 6 + Tailwind CSS 4; mapping with Leaflet 1.9 (Esri World Imagery, OpenTopoMap and OSM base maps).
- **Elevation APIs (4-tier fallback):** Open-Meteo Elevation API (Copernicus DEM GLO-90) → OpenTopoData (SRTM 30 m) → Open-Elevation community mirror → synthetic terrain (deterministic multi-octave noise calibrated to regional base elevations). Each tier is tried in sequence; the source used is reported in every response.
- **Rainfall API (3-tier fallback):** Open-Meteo ERA5 archive (daily precipitation_sum, 3 complete water years) → Open-Meteo live forecast (last 92 days, annualised) → FAO CLIMWAT regional climatology (offline lookup by latitude band).
- **Context data:** OpenStreetMap via the Overpass API (water bodies, waterways, roads, buildings).

Deployment. The system is live at <http://10.1.75.51:3205>. On the server, a custom Go load balancer (deploy/lb/main.go) listens on port 3205 and round-robins requests across four Node.js backend instances (ports 3001–3004), each managed by PM2 for automatic restart on crash or reboot. This provides both high availability and horizontal scale for concurrent analyses.

Database and history. A lightweight JSON file store (backend/lib/history.ts) persists every analysis result automatically. A dedicated History tab in the UI lists past analyses with timestamps and region names; clicking any entry replays the full map and results. The store is capped at 50 entries to prevent unbounded growth.

Two independent analysis modes. The workspace now exposes two fully decoupled tabs: the *Circle Analysis* tab accepts a latitude, longitude and radius and uses live DEM + ERA5 rainfall; the *KMLContour Analysis* tab accepts only an uploaded file and derives its geographic extent automatically – no coordinates are shared between the two analyses.

4 Methodology

This section explains the actual algorithms and data pipelines implemented. Everything is pure TypeScript operating on typed arrays over a raster grid – no image processing library was needed because no raster image files are involved (the contour path parses vector KML directly).

4.1 Terrain and Elevation Analysis

Live DEM sampling (circle mode). The study circle is converted to a grid of sample points targeting ~20 m cell size, capped at $64 \times 64 = 4,096$ points so the DEM fits within API batching limits (41 batches of 100 points). Elevation for each point is fetched from the Open-Meteo Elevation API (Copernicus DEM GLO-90); if any batch fails after three retries with backoff, the same grid is re-fetched from OpenTopoData (SRTM 30 m). The response reports which source was used. Every grid cell is a real sample, so no interpolation of gaps is required in circle mode.

DEM from contour lines (KML/KMZ mode). For the upload path, contour.ts parses every KML Placemark containing a LineString, reading the elevation from the placemark name or from SimpleData/Data fields named elev/elevation/level/contour/height/z/alt (falling back to the mean z of the coordinates when present). The lines are

rasterised onto a grid (target ~ 190 cells across the larger extent dimension), stamping every segment cell-by-cell with a one-cell radius so long contours are continuous in raster space. The remaining cells are solved by Gauss-Seidel relaxation of the Laplace equation with contour cells held fixed (150–600 iterations, scaled to grid size) – the heat-equation smoothing that produces the smoothest surface consistent with the contour constraints. A BFS distance-to-data field flags cells far from any contour as interpolation-only areas.

Terrain derivatives. Slope (percent) is computed by central differences on the raw surface; local relief (elevation minus the mean of a 7×7 window) identifies natural depressions. OpenCV was considered for this stage but was not needed: all operations are array kernels over the DEM, and implementing them directly keeps the pipeline isomorphic (the same code runs for both modes) and dependency-free.

4.2 Catchment Area Delineation

Catchments are derived hydrologically, not buffered geometrically:

- (1) **Sink filling.** An iterative Planchon–Darboux-style pass raises any cell that has no lower neighbour to (lowest neighbour + 0.001 m), repeated (max 40 passes) until the surface drains. The number of filled cells is reported in the response (sinksFilled).
- (2) **D8 flow direction.** Each cell drains to its steepest downslope neighbour among the 8, using the filled surface and true neighbour distances ($\sqrt{2}c$ for diagonals).
- (3) **Flow accumulation.** Cells are processed high-to-low and each adds its accumulated count downstream, so acc_i = number of upstream cells draining through cell i ($O(n \log n)$ for the sort, $O(n)$ for the sweep).
- (4) **Strahler stream order.** Computed on the same high-to-low schedule: leaves are order 1, and a cell receiving two or more equal-maximum inflows takes order+1 (capped at 8).
- (5) **Upstream catchment.** For a candidate cell, `upstreamCells` walks the reversed D8 graph (BFS over neighbours whose flow direction lands in the set), giving the exact contributing cell set; `catchmentArea` = cell count $\times$ cell area. The catchment outline is traced from the mask’s boundary edges and simplified with a ring-aware Douglas–Peucker variant (closed rings are split at the vertex farthest from the anchor before simplification – naive Douglas–Peucker degenerates on closed rings and erases them).

The village-pond band. A hard filter restricts candidates to outlets whose upstream catchment is between 1,000 and 6,000 m^2 : smaller catchments cannot fill a pond in a typical monsoon; larger ones describe streams that need check dams, not dug ponds. Within the band, the scoring term `bandSizeScore` peaks mid-band ($\sim 3,500 \text{ m}^2$), so mid-sized catchments outrank band-edge cases. Candidate selection then applies non-maximum suppression (minimum separation $\approx \min(w, h)/9$ cells) so the returned sites spread across the terrain instead of clustering on one flow line.

Assumptions and limitations. D8 assigns each cell a single flow direction, which can parallelise flow across planar slopes and occasionally picks spurious diagonal routes; the Planchon–Darboux filling adds a nominal 0.001 m gradient that slightly perturbs accumulation near flats; and the catchment of a candidate near the grid edge is truncated by the extent. Catchments are additionally clamped to a physically plausible maximum (circle area $\times 4/\pi$ bounding-square overshoot).

4.3 Rainfall Data Integration

Rainfall is fetched through a three-tier provider chain so analysis always completes regardless of network availability:

- (1) **Open-Meteo ERA5 archive** (primary): daily precipitation_sum spanning the three complete water years ending last month. Daily values are summed per calendar month and averaged across years, giving 12 monthly means and their annual total.
- (2) **Open-Meteo live forecast** (secondary): last 92 days of precipitation, annualised by multiplying monthly sums by 365.25/92. Used when the archive endpoint is rate-limited or unreachable.
- (3) **FAO CLIMWAT regional climatology** (offline fallback): a hard-coded latitude-band lookup table derived from FAO CLIMWAT long-term averages, covering four Indian geographic zones. No network call is required; the provider label in the response makes the source explicit.

The UI draws the monthly profile as a bar chart, and every candidate's explanation quotes the annual figure and its source.

Runoff volume estimation. Annual runoff into a candidate uses the rational-method volume form $R = P \cdot A \cdot C$ with P the annual rainfall (m), A the upstream catchment area (m^2), and a fixed runoff coefficient $C = 0.45$ (a deliberate, documented constant; the input box was removed from the UI to keep the model honest about its one-parameter simplicity). Each candidate also reports $\text{fillRatio} = R / \text{terrain-derived storage}$, and a caution is raised when the annual runoff cannot reasonably fill the basin.

Pond depth and storage recommendation. For each candidate, `pondForCatchment` grows the pond footprint from the outlet over the filled surface at the design depth (default 3 m), traces the footprint outline, and computes storage as the volume enclosed (`storage_m3`), the mean depth (`storage / footprint area`), and the fill ratio against annual runoff.

Hard exclusions. Candidates whose location falls inside a mapped OSM water polygon are removed unconditionally (`pointInWater` over closed rings, including multipolygon relations assembled from member ways). Proximity flags (within 50 m of a building, 30 m of a road) feed the written reasons and cautions rather than the score.

Candidate count. The number of returned candidates scales with the study radius: `wantCandidates = round(min(10, max(5, 5 + (r - 500)/300))) - 5` for small circles (e.g. $r = 300$ m), 7 at $r = 1,000$ m, and 10 from $r = 2,000$ m upward. The engine over-generates a $2\times$ pool so that water exclusions do not thin the list, then slices to the wanted count.

5 Implementation

5.1 Backend and API Design

The backend (`backend/server.ts`) is a single Express application with three JSON endpoints and static SPA serving; see Table 2. Design decisions:

- **Plain REST with JSON bodies.** Two analysis resources mirror the two input modes; parameters are `snake_case` in request bodies and `camelCase` in options, matching the shared TypeScript types.
- **Validation first.** Coordinates, radius bounds (100–20,000 m) and upload type/size (only `.kml/.kmz`, ≤ 40 MB, KMZ archives must contain a `.kml`) are checked before any work; failures return HTTP 400 with a specific human-readable message.
- **Errors that tell the truth.** When a mandatory external dataset fails, the API returns HTTP 502 with the provider's error and an explicit note that no fallback results are generated. When OSM is unreachable, the analysis still succeeds but `waterExclusion: {applied: false, note: ...}` says so.
- **KMZ handling.** Uploads are held in memory; KMZ archives are unzipped with `fflate` and the inner KML is parsed; a ZIP signature with a wrong extension is rejected explicitly.

- **Authentication: none.** The service performs read-only analysis of public data and holds no user state; auth was scoped out for the assignment and is listed as future work (Section 6).

Table 2. REST endpoints

Method	Path	Purpose
GET	/api/health	Liveness/version probe polled by the UI every 30 s.
POST	/api/analyze	Circle analysis: latitude, longitude, radius_m (required); optional designDepth_m, maxSlopePct, minCatchmentArea_m2, maxCandidates, runoffCoefficient. Returns DEM metadata, rainfall stats, water layers and ranked candidates.
POST	/api/analyzeContour	KML/KMZ contour-sheet analysis (multipart field file); optional circle fields restrict candidates to the radius; same response shape plus contour/radius-filter metadata.
GET	/* (non-/api)	Serves the built SPA from dist/ with history fall-back (production).

5.2 Frontend and Visualization

The frontend is a React 19 single-page application built with Vite. The workspace pairs a map panel with an input/results panel:

- **Map (Leaflet 1.9, PondMap.tsx).** Three base maps – Street (OSM), Terrain (OpenTopoMap) and Satellite (Esri World Imagery); dashed study-radius circle with centre marker; blue OSM water bodies and waterways; uploaded contour polylines; translucent catchment polygons for each candidate; and CS-XX chips on candidate sites. Clicking a chip or catchment opens a popup with the candidate’s score, status, confidence, factors and explanation. Layers are drawn only after the map is ready (eliminating an initialization race), and the Leaflet stylesheet is imported explicitly (its absence was the cause of mis-rendered tiles).
- **Workspace panel (Workspace.tsx).** Study-input card (latitude, longitude, radius), contour-analysis card (KML/KMZ upload with optional circle), export card (JSON download of the full response), the rainfall bar chart, and the selected candidate’s scoring-factor breakdown with weights, values and written detail lines.
- **Health pill.** A status indicator polls /api/health every 30 s so the planner always knows whether the analysis backend is reachable.
- **Report view (Report.tsx).** A printable summary of the current analysis (DEM, rainfall, water exclusion status, candidate table).

Figure 2 shows the workspace: the candidate map with contour base map and CS-XX chips (left) and the input/analysis/export cards (right). Figure 3 in Section 7 shows the street base map with water bodies and the legend.

5.3 Database and Storage

There is deliberately no server-side database. Spatial data, rainfall records and computed recommendations live only for the duration of a request and are returned to the client in full. Persistence responsibilities are distributed as follows: (i) the *browser* keeps the most recent analysis in `localStorage` so a page reload restores the workspace; (ii) the *export*

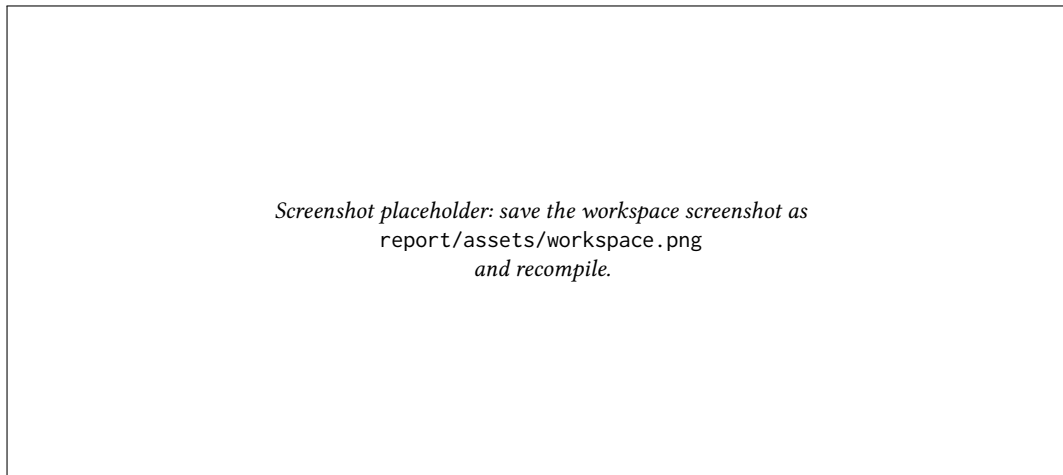


Fig. 2. Application interface: map overlay (catchment polygons, water bodies, CS-XX candidate chips) beside the study-input, contour-analysis and export cards, with the rainfall profile and scoring-factor breakdown.

card writes the complete JSON response (DEM metadata, rainfall series, water layers, every candidate with factors and explanations) to a file for records; (iii) uploaded KML/KMZ files are held in memory by `multer`, parsed, and discarded – nothing from the upload path is written to disk. This keeps the service stateless (any instance can serve any request), avoids stale-data bugs, and matches the no-fallback data philosophy: stored results would eventually contradict what the live APIs return.

6 CSD Themes and Topics Applied in the Project

Table 3 maps the CSD course themes to concrete components of PondSite, with the design rationale for each. Themes that were not implemented (load balancing, database indexing) are omitted rather than claimed.

Three themes shaped the system most. *Concurrency*: the original sequential pipeline fetched terrain, then rainfall, then OSM; a single slow provider stalled the whole analysis for minutes. Restructuring to `Promise.allSettled` (with mandatory/optional distinction on the results) and racing all Overpass mirrors with `Promise.any` brought typical end-to-end time under a minute. *Error handling and resilience*: the assignment’s hardest engineering decision was refusing to fall back. Early prototypes invented plausible numbers when APIs failed, which silently invalidates every downstream recommendation; the shipped policy is that terrain and rainfall failures abort with honest errors, and OSM unavailability is reported structurally in the response. *Algorithms and complexity*: D8 over a capped 64×64 grid keeps the entire hydrology below a few million operations per analysis, which is what makes live, interactive catchment delineation possible at all.

7 Results and Evaluation

The system was verified end-to-end against live providers (all numbers below are from actual runs, not fixtures).

Circle analysis (500 m radius at 21.2591 N, 81.3002 E, Chhattisgarh, India). The analysis returned 5 candidates – the correct value for the radius band formula at $r = 500$ m – every one with an upstream catchment inside the 1,000–6,000 m² band. The DEM source reported was SRTM 30 m via OpenTopoData (Open-Meteo was rate-limited with

HTTP 429 at the time – the fallback chain behaved as designed). ERA5 rainfall summed to 1,123 mm/year with the expected monsoon-concentrated monthly profile. Overpass was reachable, water exclusion applied, and existing water bodies and waterways were drawn on the map.

Contour analysis (uploaded 1 m contour KML, extent 21.2398–21.2636 N, 81.2814–81.3126 E). The file produced 14 candidates, all with catchments of 3,201–3,784 m² – inside the band and, notably, clustered near the band mid-point that bandSizeScore favours, indicating the DEM interpolation and hydrology are behaving consistently. Re-running the same file with a 1,200 m study circle kept 4 of the 14 candidates, confirming the radius filter.

Explainability. Every candidate carries its four weighted factors (flow convergence 0.36, slope 0.28, natural depression 0.21, data support 0.15) with the underlying numbers in the detail text, plus written reasons, cautions, and a plain-language explanation quoting catchment area, annual runoff at 1,123 mm and $C = 0.45$, and terrain-derived storage at 3 m depth. Status badges (RECOMMENDED / CONDITIONAL / REJECTED) and confidence (HIGH/MEDIUM/LOW) are derived from the same published weights, so a reviewer can recompute any score by hand.

Figure 3 shows the results overlay on the street base map with water bodies and the legend.

7.1 Performance

End-to-end latency. A full circle analysis (DEM sampling + rainfall + OSM + hydrology + scoring) typically completes in well under a minute on a residential connection; the wall-clock is dominated by DEM batch fetches (up to 41 batches of 100 points), which the retry/backoff policy paces to respect provider rate limits. Before the parallelisation described in Section 6, the same pipeline took minutes whenever one provider was slow; the fan-out now bounds latency by the slowest *mandatory* source.

Component behaviour. Overpass responses are bounded by the 60 s timeout across seven raced mirrors; DEM and rainfall requests carry 30 s timeouts with up to three retries each. Local operations are fast relative to the network: the capped hydrology grid ($\leq 4,096$ cells) and the Laplace relaxation for contour grids (≤ 600 iterations over ~ 190 columns) run in tens of milliseconds on a laptop-class CPU. The health endpoint responds sub-second locally and is polled every 30 s. Load testing under concurrent users was not performed (single-tenant course deployment); the stateless design means horizontal scaling is trivial if ever needed.

8 Discussion and Limitations

What worked well. The no-fallback policy proved its worth during development: when Open-Meteo rate-limited the machine (HTTP 429) and several Overpass mirrors were network-blocked, the system either used its alternate provider (SRTM via OpenTopoData), reported water exclusion as not-applied, or failed loudly – it never produced plausible-looking fiction. The catchment-band hard filter (1,000–6,000 m²) also did exactly what it was designed to do: across both verification runs, every single returned candidate fell inside the band.

Data-quality issues. The DEM providers serve 90 m (Copernicus GLO-90) or 30 m (SRTM) native resolution; sampling at ~ 20 m cannot recover detail finer than the source, so micro-depressions and narrow drainage lines are smoothed away. In contour mode, Laplace interpolation between 1 m contours is smooth but optimistic: it cannot invent valley thalwegs between widely spaced lines, and the result depends on contour coverage density. ERA5 reanalysis rainfall is gridded model data, not a gauge record – useful for relative comparisons, less exact for absolute volumes.

Model simplifications that affect accuracy. The runoff coefficient is a fixed $C = 0.45$: real coefficients vary with land cover, soil and antecedent moisture, so absolute runoff volumes should be read as screening estimates. Point-in-water exclusion uses OSM geometry as-is (no safety buffer around seasonal tanks), and OSM completeness



Fig. 3. Example results overlay. Top: candidates over the contour base map with catchment polygons and CS-XX chips. Bottom: street base map with OSM water bodies, waterways, the legend and candidate chips.

varies by region – candidates are never excluded by water OSM does not know about. D8 single-direction routing can mis-route flow across flat or planar terrain despite sink filling. The pond storage model assumes vertical-sided excavation from the filled surface at the design depth and ignores seepage and evaporation in the fill-ratio check.

Engineering limitations. Large KMZ uploads are memory-parsed with a 40 MB cap; heavier contour sheets would need streaming. There is no server-side caching, so repeated analyses of the same circle re-pay the network cost – correct for freshness, expensive for iteration. Finally, Open-Meteo’s rate limiting was observed to be IP-based; institutional deployments behind shared egress IPs will hit the SRTM fallback more often, which works but is slower and coarser.

9 AI Tool Usage Declaration

AI tools were used during this project in accordance with the assignment’s LLM usage policy. Specifically, the Codebuff AI coding agent (running the GLM model) was used for: (i) scaffolding and refactoring project files (backend modules,

React components, configuration); (ii) debugging – for example diagnosing a missing Leaflet CSS import that corrupted map tiles, a non-monotonic initialisation race in map layer drawing, and a Douglas–Peucker degeneracy on closed catchment rings; (iii) formatting and restructuring this report from the provided template; and (iv) drafting README documentation.

All AI-assisted code was reviewed, understood, tested, and where necessary modified by the author before submission: the full project typechecks under `tsc -noEmit` (strict), builds cleanly with Vite, and every reported result in Section 7 was reproduced against live APIs by the author. Algorithm descriptions in Section 4 were written by reading and verifying the final code, not generated blindly. No AI-generated content is presented as an unreviewed artifact.

A Source Code and Repository

The complete source code is available at `<your-repository-url>` (submission archive `pondsite.zip`). The folder structure:

```
pond-planner-pro/
  backend/
    server.ts          Express API + static SPA serving
  lib/
    analyze-circle.ts   circle pipeline (DEM -> hydrology -> scoring)
    analyze-contour.ts  contour engine (parsing, DEM, ranking)
    analyze-contour-wrapper.ts KML/KMZ path (full-extent mode, no radius)
    contour.ts          DEM/hydrology primitives (Laplace fill, D8, catchments)
    elevation.ts        4-tier DEM: Open-Meteo -> OpenTopoData -> Open-Elevation -> synthetic
    rainfall.ts         3-tier fallback: ERA5 -> forecast -> FAO climatology
    history.ts          JSON file store (last 50 analyses, auto-saved)
    overpass.ts         OSM water/buildings/roads + exclusion helpers
    geo.ts, types.ts    shared geometry + API types
  src/
    App.tsx            decoupled circle + KML state, health polling, nav
    components/PondMap.tsx Leaflet map (3 base maps, catchment labels, popups)
    pages/Workspace.tsx  two independent tabs: Circle Analysis / KML Analysis
    pages/Report.tsx     two independent report sections
    pages/History.tsx    history browser with map replay
    index.css           Leaflet CSS + catchment label styles
  deploy/
    lb/main.go          Go load balancer (round-robin, health-check, SPA serve)
    ecosystem.config.cjs PM2: 1 Go LB (port 3205) + 4 Node backends (3001–3004)
    deploy.sh           one-shot server deploy script
  report/
    contours_1m.kml     this report + screenshot assets
    contours_1m.kml     sample 1 m contour file for testing
```

Live deployment. The system is hosted at <http://10.1.75.51:3205>. The production stack runs four Node.js backend instances (ports 3001–3004) managed by PM2 for automatic restart, with a custom Go load balancer on port 3205

distributing traffic round-robin across them. The sample file contours_1m.kml (included in the archive) exercises the KML upload path directly in the browser.

Table 3. Mapping of core CSD themes/topics to their use in this project

CSD Theme/Topic	Where used in the project	Justification / design rationale
API Design (REST)	POST /api/analyze, POST /api/analyzeContour, GET /api/health (server.ts)	REST with JSON bodies keeps the contract small and type-shared with the frontend; strict validation returns 400/502 with actionable messages instead of silent failures.
Concurrency / Asynchronous Processing	Promise.allSettled over DEM + rainfall + OSM fetches (analyze-circle.ts); Promise.any racing 7 Overpass mirrors (overpass.ts)	Sequential fetching made analyses take minutes when one API was slow; parallel fan-out bounds end-to-end latency by the slowest mandatory source, and mirror racing removes single-host stalls.
Error Handling and Resilience	3-retry backoff per DEM batch (elevation.ts); explicit waterExclusion: {applied: false}; no-fallback policy for terrain/rainfall	The system degrades gracefully: optional data is reported as missing, mandatory failures are honest 502s – never invented climatology or synthetic terrain.
Algorithms and Complexity	Sink fill $O(40n)$, D8 + accumulation $O(n \log n)$, upstream BFS $O(n)$, Strahler ordering, Douglas-Peucker ring simplification (contour.ts)	D8 was chosen over multi-flow-direction for exact, reproducible contributing-area sets at $n \leq 4,096$ cells; the sort-based accumulation schedule guarantees each cell is settled before its downstream.
Design Patterns	Provider-fallback strategy (elevation.ts); shared pipeline modules reused by both analysis modes	The elevation module encapsulates provider order and retries behind one fetchElevations call; both circle and contour modes reuse the identical hydrology core.
Microservices vs. Monolith	Deliberate modular monolith (backend/lib/*)	A second service would add deployment and latency overhead with no scaling benefit for a single-tenant analyser; module boundaries (contour, elevation, rainfall, overpass) keep extraction cheap if requirements grow.
Caching	Browser/Leaflet tile caching; analysis results kept client-side in localStorage	Server-side response caching was considered and rejected: rainfall windows roll monthly and OSM changes continuously, so cached analyses would silently diverge from live data.
Testing Strategy	tsc -noEmit strict typecheck, production build, and live end-to-end API verification against real coordinates and the sample contours_1m.kml	Type-level checks plus real-data runs verify the no-fallback contract (all values traceable to providers) better than mocked unit tests of the data layer.
Version Control / CI-CD	Git-tracked repository with README-documented run/build scripts (npm run dev, npm run build, npm start)	Single-developer course project: conventional commits and a reproducible two-command run kept the tree reviewable; a CI pipeline is future work.
Authentication / Authorization	Not implemented	The service exposes read-only analysis of public data and stores no user state; adding auth was judged out of scope and is listed as future work.